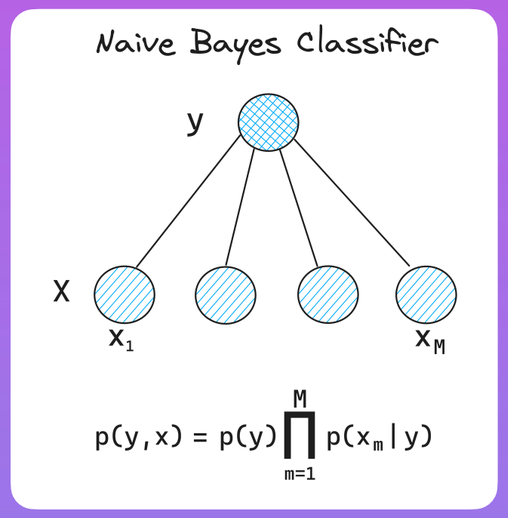
Credit Risk Modeling with Naive Bayes

Introduction to Naive Bayes



Naive Bayes is a set of supervised learning algorithms used for classification based on applying Bayes' theorem with the "naive" assumption of conditional independence between features

Naive Bayes classifiers differ mainly by the assumptions they make regarding the distribution of P(X | y)

Objective:

to estimate Y given the values of X_i's or

to estimate $P(Y|x_1,x_2,...x_m)$ using Naive Bayes

Main assumption (a very bold one!):

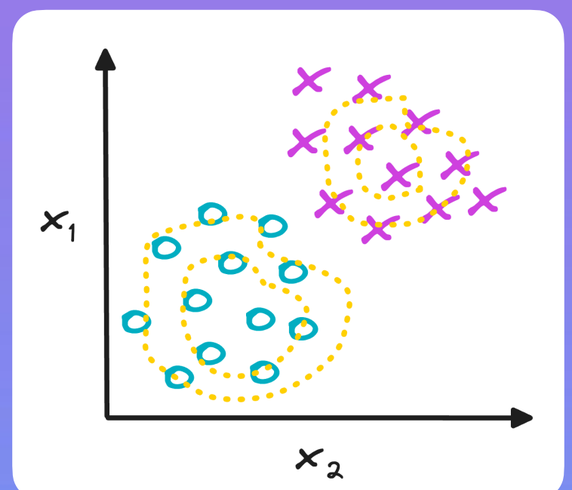
All X_i's are independent given the label!

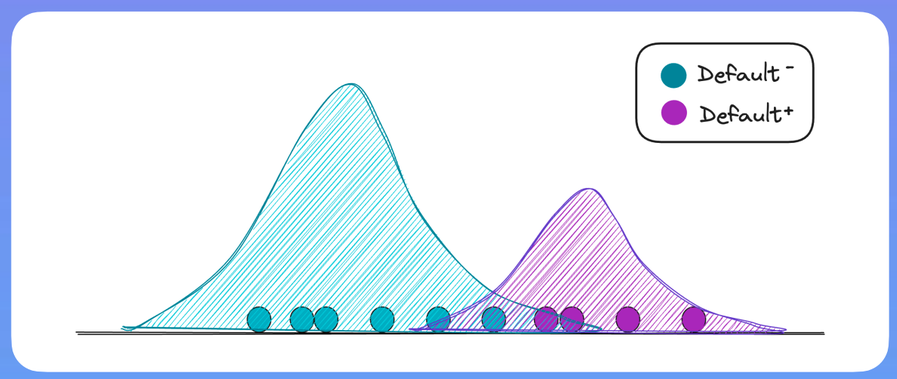
With Naive Bayes, we are building a model of what data in each class looks like (i.e., the distribution of the data)

We model p(X|y) for each of value of y and p(y) and then use Bayes' rule to find p(y|X)

The reason why it belongs to the group of generative models (vs discriminative like Logistic Regression, Boosting etc.)

"All models are wrong, but some are useful" –George P. Box: Naive Bayes is very useful in credit scoring problems





Depending on the distributions of input features, different assumptions are made about estimating the parameters of Naive Bayes:

- For continuous features, the likelihood of the features is assumed to be Gaussian (shown above)
- For categorical features, Naive Bayes estimates a categorical distribution for each feature i of vector X conditional on the class y (how WOE works)

Weight-of-Evidence (WOE) primer

	Default⁺	Default⁻
past delinquency +	185	200
past delinquency -	15	800
Totals	200	1000

We can consider this 2x2 contingency table showing the relationship between a past delinquency event and the default flag

If we represent the table as conditional probabilities, we can see that 93% of customers who had a delinquency eventually defaulted

	Default⁺	Default⁻	WOE
past delinquency +	92.5%	20%	-1.53
past delinquency -	7.5%	80%	+2.36
Totals	100%	100%	-1.61

We can easily calculate a Weight-of-Evidence (WOE) score from this table

WOE is a natural log of the ratio between % goods (number of default- in rows to total default+) and % bads (number of default+ in a rows to total default+)

The WOE value -1.61 in row Totals is the log odds of the defaulters to non-defaulters for this problem (200 / 1000 = 0.2, log-odds = ln(0.2) = -1.61

So if we want to calculate the probability of default given a past delinquency+, it's very simple by summing the intercept (-1.61) and the WOE score (-1.53) and then converting into probability using a sigmoid function:

- Logit score = -1.61 + (-1.53 * -1) = -0.078
- PD = 1 / (1 + exp(-(-0.078)) = 48.5%

	Default⁺	Default⁻	Default rate
past delinquency +	185	200	48.5%
past delinquency -	15	800	1.8%
Totals	200	1000	16.6%

We can confirm this calculation by looking at the row with past delinquency+:

185 / (185+200) = 48.5% in the first row of the table

The Naive Bayes formula can we rewritten into the log-odds format using WOE:

$$p(y,x) = p(y)\prod_{m=1}^{M} p(x_m \mid y)$$

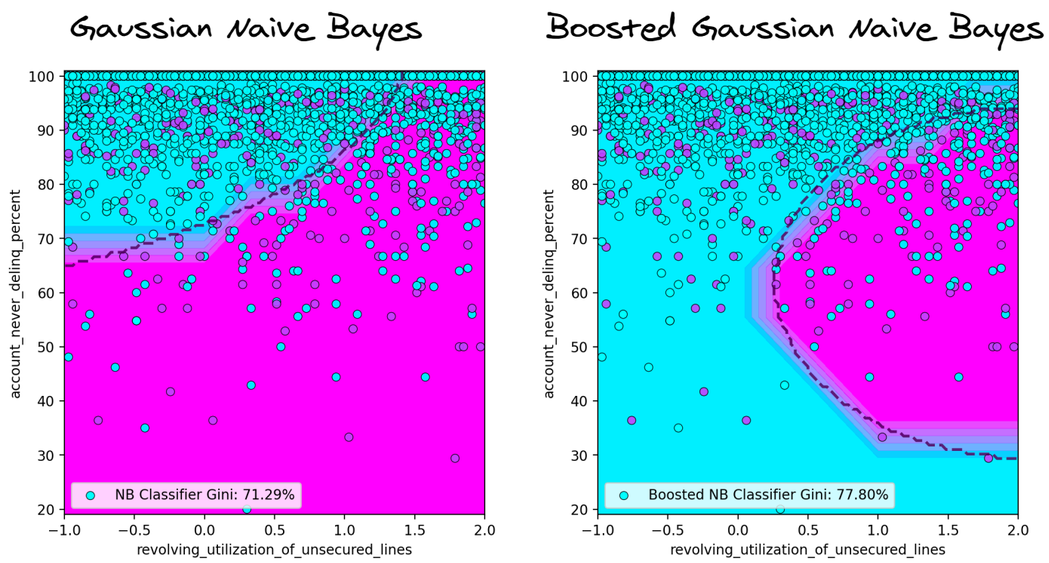
$$= \sigma\left(\log\frac{P(Y=1|X_m)}{P(Y=0|X_m)} = \underbrace{\log\frac{P(Y=1)}{P(Y=0)}}_{\text{Sample log-odds}} + \underbrace{\log\frac{f(X_m|Y=1)}{f(X_m|Y=0)}}_{\text{WOE}}\right)$$

The expression above says that the probability of default equals the sum of the WOE vectors and sample log-odds converted to probability with the sigmoid function

Credit Risk Modeling with Naive Bayes

Naive Bayes models

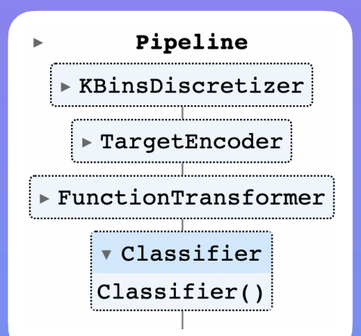
To illustrate some examples of Naive Bayes modeling, we can visualize decision boundary plots of different Naive Bayes models for the two features, past delinquency and credit card utilization

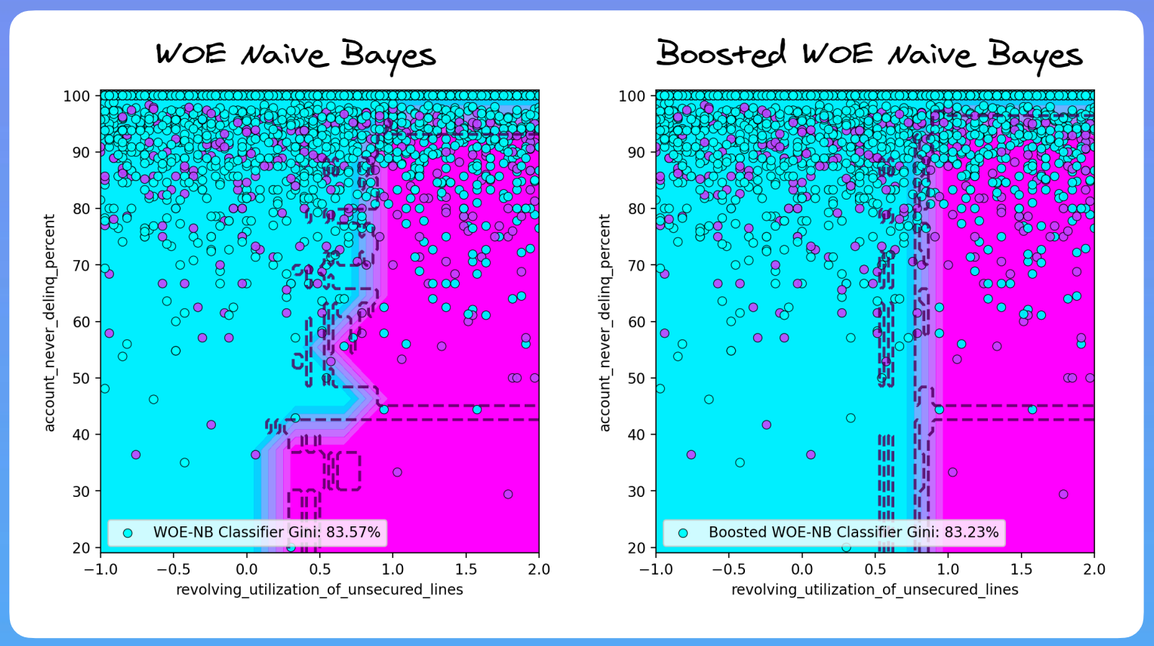


We can observe a noticeable difference in the Gini scores, where boosting of Naive Bayes allows to increase the Gini score by +7%

To improve the performance further can employ a "pseudo-WOE" approach, which is based on the following:

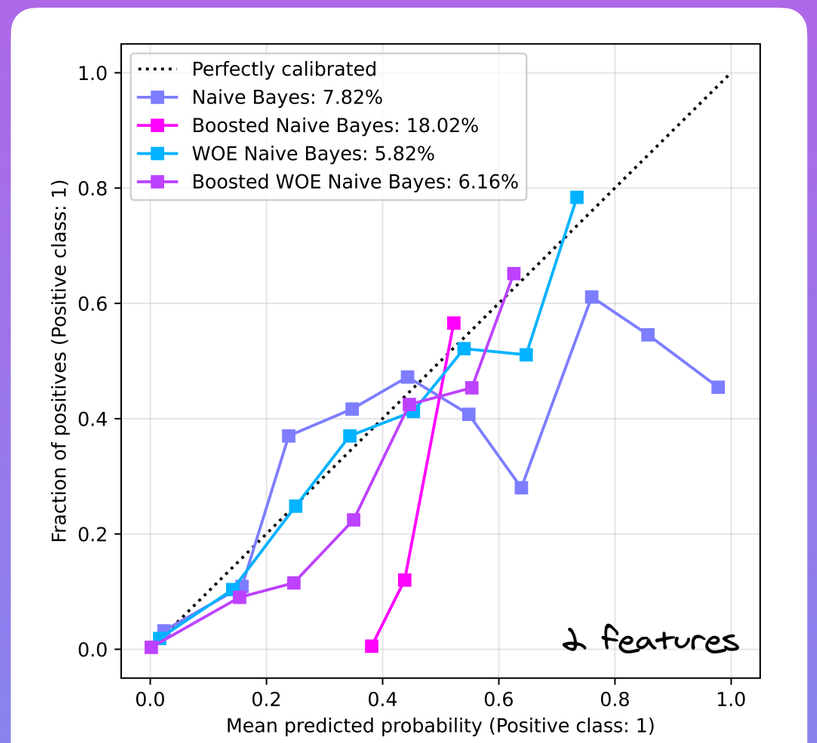
- Binning of features (e.g., 30 bins)
- Target-encoding of features
- Converting the encoded probabilities to log-odds (logits)
- Subtracting sample log-odds from logits to get "pseudo-WOE" scores (derived backwards)
- Fitting a classifier (using a sigmoid function or AdaBoost)



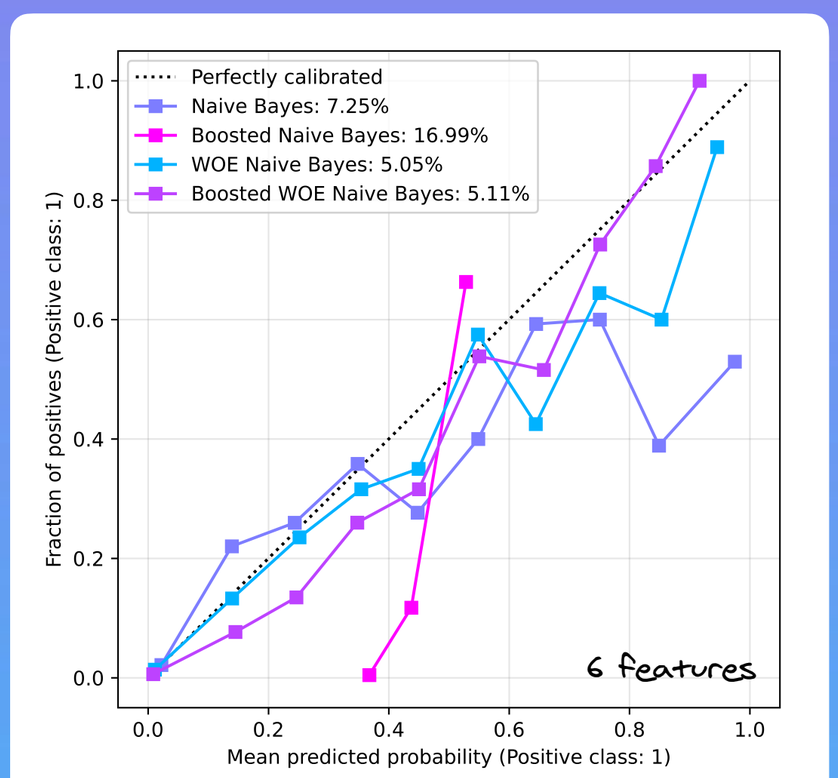


Calibration

We can further assess the calibration of different models shown previously to get an intuition of how our predictions fare against the observed outcomes also reporting Brier scores for each model



The best calibrated score is WOE Naive Bayes, which also has the highest Gini score. We repeat this experiment with adding several more features, the results are very similar



Credit Risk Modeling with Naive Bayes

NOTEBOOK

Pseudo-WOE Naive Bayes Pipeline

```python
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import (
    KBinsDiscretizer,
    TargetEncoder,  # type: ignore
    FunctionTransformer
)
from sklearn.pipeline import Pipeline

import warnings
warnings.filterwarnings("ignore")

base_log_odds = np.log(np.mean(y.loc[ix_train]) / (1 -
np.mean(y.loc[ix_train])))
# This means we take average DR in bin and convert to log-odds like intercept
# After this we subtract the intercept to create WOE scores
def convert_to_woe(X: np.ndarray):
    X = np.clip(X, 1e-5, 1 - 1e-5)
    # we get log odds first
    X = np.log(X / (1 - X))
    # then we subtract the base log odds
    X = X - base_log_odds
    return X

# create an Naive Bayes pipeline
nb_pipe_woe_clf = Pipeline(
    [
        (
            "discretizer",
            KBinsDiscretizer(
                n_bins=30,
                encode="ordinal",
                strategy="kmeans",
                random_state=0
            ).set_output(transform="pandas"),
        ),
        (
            "encoder", TargetEncoder(smooth=1.0, random_state=0)),
        (
            "woe", FunctionTransformer(convert_to_woe, validate=False)),
        (
            "sigmoid_classifier",
            Classifier(),
        ),
    ]
)
```

Sigmoid Classifier

```python
from sklearn.base import BaseEstimator, ClassifierMixin
import numpy as np

class Classifier(BaseEstimator, ClassifierMixin):
    def __init__(self):
        self.base_log_odds = 0  # Initialize base log odds
        self.is_fitted_ = False  # Add an attribute to track
if the model is fitted

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def fit(self, X, y):
        self.base_log_odds = np.log(np.mean(y) / (1 -
np.mean(y)))
        self.is_fitted_ = True  # Set is_fitted_ to True
after fitting
        return self

    def predict_proba(self, X):
        if not self.is_fitted_:
            raise NotFittedError("This Classifier instance
is not fitted yet. Call 'fit' with appropriate arguments
before using this estimator.")

        predicted_log_odds = X.sum(axis=1) +
self.base_log_odds
        prob_positive_class =
self.sigmoid(predicted_log_odds)
        return np.column_stack((1 - prob_positive_class,
prob_positive_class))

    def predict(self, X):
        predicted_proba = self.predict_proba(X)
        return np.argmax(predicted_proba, axis=1)
```